

FocusVision Salient Object Detection using Deep Learning

Dataset : ECSSD Saliency Detection Dataset

Framework: PyTorch, Google Colab, CUDA GPU Acceleration

Project Type: Computer Vision / Image Segmentation Project

Core Techniques: CNN Encoder-Decoder Architecture, BCE + IoU Loss, Skip Connections, Batch Normalization

Author

Olti Miftari

1. Introduction

Salient object detection is a computer vision task that focuses on identifying and segmenting the most visually important object in an image. The main objective is to generate a saliency mask that separates the foreground object from the background.

This project explores the implementation of multiple convolutional neural network (CNN) architectures for salient object detection using PyTorch. The project was developed as part of the GigaAcademy Machine Learning training program.

The complete pipeline includes:

- 1.Dataset preparation
- 2.Image preprocessing
- 3.Data augmentation
- 4.CNN model implementation
- 5.Model training and validation
- 6.Evaluation using segmentation metrics
- 7.Prediction visualization
- 8.Architecture experimentation and comparison

The project was implemented and trained using Google Colab with GPU acceleration.

2. Problem Statement

The objective of this project is to build an end-to-end deep learning system capable of detecting and segmenting the most visually important object in an image.

The generated output should be a binary saliency mask where:

White regions represent the salient object

Black regions represent the background

The project focuses on experimenting with custom CNN architectures instead of relying on pretrained segmentation frameworks.

3. Dataset

3.1 ECSSD Dataset

The project uses the ECSSD (Extended Complex Scene Saliency Dataset).

The dataset contains:

-RGB images

-Corresponding ground truth saliency masks

The dataset includes complex scenes with multiple object types such as:

1. **Animals**
2. **Humans**
3. **Natural scenes**
4. **Indoor objects**
5. **Outdoor environments**

3.2 Dataset Size

Total **images**: **1000**

Total **masks**: **1000**

The dataset was split into:

80% training data

20% validation data

4. Technologies and Libraries

The following technologies and libraries were used during the project:

1. Python
2. PyTorch
3. NumPy
4. Matplotlib
5. Scikit-learn
6. Google Colab
7. Git & GitHub

5. Data Preprocessing

Several preprocessing techniques were applied before training the models.

5.1 Image Resizing

All **images** and **masks** were resized to:

224 × 224

This ensured consistent input dimensions during training.

5.2 Data Augmentation

To improve model generalization and reduce overfitting, the following augmentation techniques were used:

- Random Horizontal Flip
- Random Rotation

5.3 Tensor Conversion

Images and masks were converted into PyTorch tensors before being passed to the models.

6. CNN Architectures

6.1 Baseline CNN Model

The first model implemented in the project was a simple encoder-decoder CNN architecture.

The architecture included:

- Convolutional layers
- ReLU activation functions
- MaxPooling layers
- ConvTranspose2D decoder layers

The purpose of this model was to create a baseline segmentation pipeline and evaluate the initial performance.

6.2 Improved CNN Model

The second model introduced a combined BCE + IoU loss function.

This experiment aimed to improve:

1. Object localization
2. Segmentation quality
3. Training stability

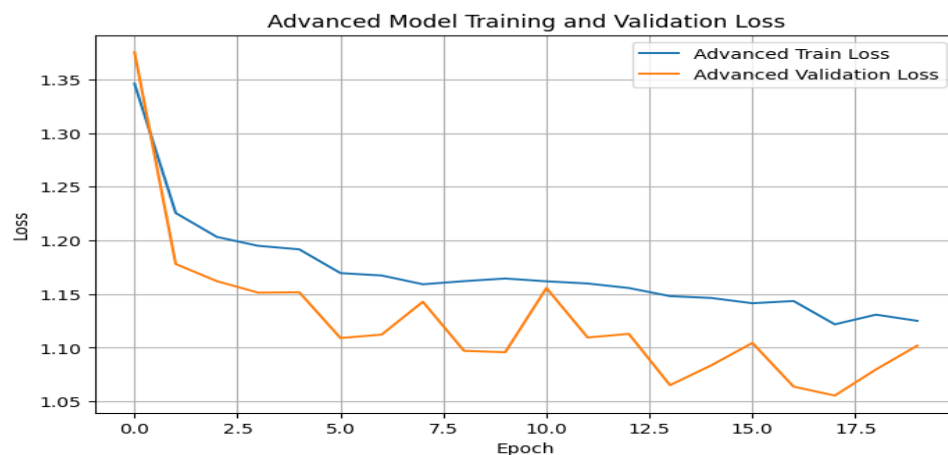
6.3 Advanced CNN Architecture

After evaluating the baseline models, a more advanced encoder-decoder architecture was implemented.

The advanced model introduced:

- Batch Normalization
- Double convolution blocks
- Skip connections

This architecture significantly improved segmentation performance and prediction quality.



8. Evaluation Metrics

The models were evaluated using multiple segmentation metrics.

8.1 **IoU** (Intersection over Union)

IoU measures the overlap between the predicted segmentation mask and the ground truth mask.

8.2 **Precision**

Precision measures how many predicted foreground pixels are actually correct.

8.3 **Recall**

Recall measures how many real foreground pixels were successfully detected.

8.4 **F1 Score**

The F1 score combines precision and recall into a single performance metric.

9. Results

9.1 Improved CNN Results

Metrics

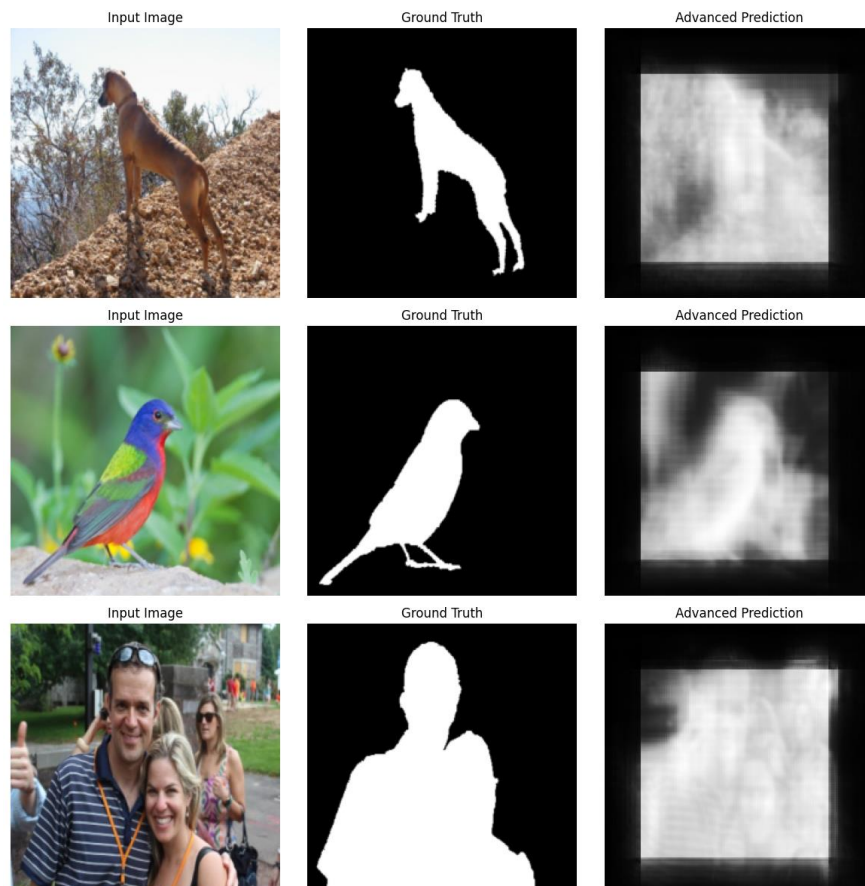
- IoU Score: 0.1738
- Precision: 0.5282
- Recall: 0.1956
- F1 Score: 0.2854

9.2 Advanced CNN Results

Metrics

- IoU Score: 0.4206
- Precision: 0.5787
- Recall: 0.6102
- F1 Score: 0.5940

The **advanced CNN architecture** produced significantly sharper and more localized segmentation masks.

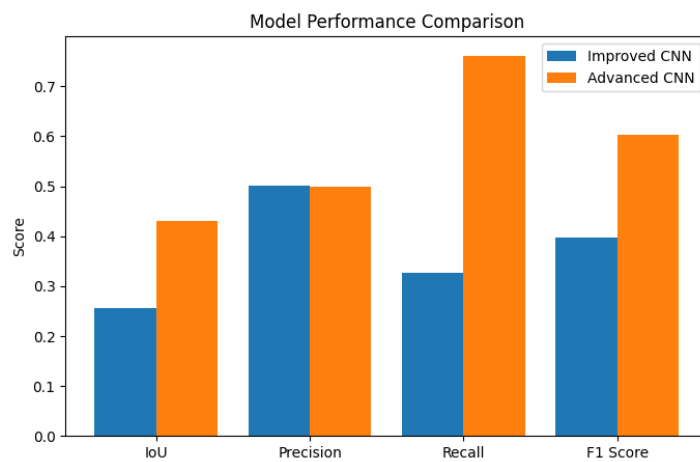


10. Model Comparison

The **advanced CNN architecture** clearly outperformed the previous models.

Main **improvements** included:

- ✓ Better object localization
- ✓ More accurate segmentation masks
- ✓ Improved Recall and F1 Score
- ✓ Higher IoU performance

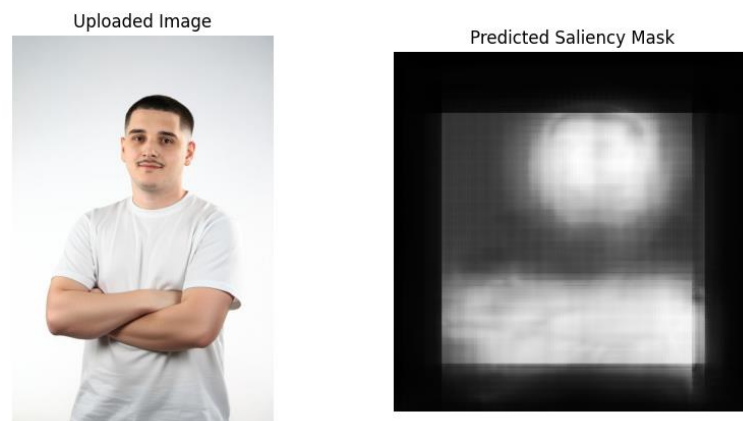


11. Inference Performance

The final advanced model achieved an average inference time of:

- 0.0067 seconds

This indicates that the model is lightweight and capable of generating predictions relatively quickly.



12. Challenges Faced

Several **challenges** were encountered during the project:

- 1) Google Colab runtime resets
- 2) Dataset extraction and file management
- 3) Training stability during early experiments
- 4) Improving segmentation quality without pretrained models
- 5) Balancing model complexity and training time

The project was **gradually improved** through experimentation and iterative testing.

13. Learning and Conclusions

This project provided practical experience in deep learning, computer vision, and image segmentation using PyTorch. One of the most important lessons learned during the project was the impact of architecture design on segmentation quality. The advanced encoder-decoder CNN architecture with skip connections and Batch Normalization significantly improved both visual predictions and evaluation metrics. Another important learning point was the effect of combining BCE and IoU loss functions. BCE helped optimize pixel-level predictions, while IoU improved the overall overlap quality between predicted masks and ground truth masks.

The project also demonstrated the importance of:

- ✚ experimentation
- ✚ model comparison
- ✚ training visualization
- ✚ evaluation metrics
- ✚ data preprocessing
- ✚ architectural improvements

Overall, the project successfully implemented an end-to-end salient object detection pipeline capable of generating saliency masks from input images.

14. Future Improvements

Possible future improvements include:

Training on larger datasets, Increasing the number of training epochs, Using pretrained encoders, Implementing attention mechanisms, Experimenting with U-Net or DeepLab architectures, Hyperparameter tuning, Testing additional augmentation techniques

